LESSON 3

Managing Application State, Persistent Data, Documents & Knowledge Bases

Practical project: SmartBudget JA - Personal Financial Management and Knowledge Retrieval Application

Your goal: Build a multi-screen Streamlit application that records income and expenses, calculates balances, saves information, uploads financial documents, extracts text, maintains a knowledge base and retrieves answers. Work through the instructions in order. Use fictional or redacted documents only.

Part 1 - Application State

Application state is the information an application keeps about its current condition or your interactions. Streamlit normally reruns your Python script when you interact with a widget. You will compare an ordinary variable with st.session_state.

1.1 Ordinary variables

Your task: Create state_demo.py in VS Code and enter the complete code below.

```
import streamlit as st
st.title("State Demonstration")
counter = 0
if st.button("Increase Counter"):
    counter = counter + 1
st.write("Counter:", counter)
```

Run in your terminal

```
python -m streamlit run state_demo.py
```

Your task: Click Increase Counter several times and record what happens.

Think and discuss: Why does the counter return to 1 instead of increasing to 2, 3 and 4?

1.2 Session state

Session state retains values during your active Streamlit session. Unlike a database or saved file, it is not reliable permanent storage.

Your task: Replace the complete contents of state_demo.py with the following code, then run it again.

```
import streamlit as st
st.title("Session State Demonstration")
if "counter" not in st.session_state:
    st.session_state.counter = 0
if st.button("Increase Counter"):
    st.session_state.counter += 1
st.write(
    "Counter:",
    st.session_state.counter
)
```

Your task: Click Increase Counter repeatedly. Observe the difference.

1.3 Reset exercise

Your task: Add the following block to your program. Demonstrate that you can increase and reset the counter.

```
if st.button("Reset Counter"):
    st.session_state.counter = 0
```

Think and discuss: What causes Streamlit to rerun? Why does an ordinary counter reset? What does session state do? Will it reliably survive a server restart?

Part 2 - Data Structures and Financial Calculations

A dictionary stores named key-value pairs. A list stores multiple items. A list of dictionaries is useful for storing expense records.

2.1 One expense record

```
expense = {
    "id": "EXP-001",
    "category": "Utilities",
    "provider": "JPS",
    "amount": 15000.00,
    "month": "2026-10"
}
print(expense["provider"])
print(expense["amount"])
```

Inspect the dictionary keys such as provider, category and amount.

2.2 Multiple expenses

```
expenses = [
    {
        "category": "Utilities",
        "provider": "JPS",
        "amount": 15000
    },
    {
        "category": "Utilities",
        "provider": "NWC",
        "amount": 5000
    },
    {
        "category": "Telecommunications",
        "provider": "Flow",
        "amount": 7500
    }
]
for expense in expenses:
    print(
        expense["provider"],
        expense["amount"]
    )
```

2.3 Calculating totals and balances

```
def calculate_total_expenses(expenses):
    total = 0
    for expense in expenses:
        total += expense["amount"]
    return total
def calculate_balance(income, expenses):
    total_expenses = calculate_total_expenses(expenses)
    return income - total_expenses
```

> Think and discuss: Before executing the code, what total do you predict for the three sample expenses? How would you calculate the balance after expenses?

For the full application, monetary amounts will be stored as integer Jamaican cents to avoid binary floating-point rounding problems. For example, J$15,000.00 becomes 1,500,000 cents.

Part 3 - Build the SmartBudget JA Project

> Your task: Create a folder named smartbudget-ja and reproduce this project structure. The two __init__.py files can remain empty.

```
smartbudget-ja/
├── app.py
├── requirements.txt
├── screens/
│   ├── __init__.py
│   ├── dashboard.py
│   ├── finances.py
│   ├── documents.py
│   ├── knowledge_base.py
│   └── ask_smartbudget.py
├── services/
│   ├── __init__.py
│   ├── finance_service.py
│   ├── storage_service.py
│   ├── document_service.py
│   └── knowledge_service.py
├── data/
│   ├── finances.json
│   └── knowledge_base.json
├── uploads/
└── sample_documents/
```

We use screens/ instead of Streamlit's special pages/ directory because this project manages navigation explicitly.

3.1 Set up your environment

```
python -m venv .venv

# Windows PowerShell
.venv\Scripts\Activate.ps1

# macOS / Linux
source .venv/bin/activate
```

Your task: Create requirements.txt with the following two dependencies.

```
streamlit
pymupdf
```

Install dependencies

```
python -m pip install -r requirements.txt
```

A virtual environment isolates project packages. Streamlit provides the interface; PyMuPDF extracts embedded text from supported PDFs. Python includes JSON support without a separate package.

3.2 Finance service

Your task: Create services/finance_service.py and enter all the following code.

```
from decimal import Decimal, InvalidOperation
def to_cents(amount):
    try:
        value = Decimal(str(amount))
        if not value.is_finite():
            raise ValueError("Amount must be finite.")
        if value < 0:
            raise ValueError("Amount cannot be negative.")
        if value.as_tuple().exponent < -2:
            raise ValueError("Use no more than two decimal places.")
        return int(value * 100)
    except (InvalidOperation, TypeError):
        raise ValueError("Invalid monetary amount.")
def format_money(cents):
    return f"J${cents / 100:,.2f}"
def calculate_total_expenses(expenses):
    return sum(
        expense["amount_cents"]
        for expense in expenses
    )
def calculate_balance(income_cents, expenses):
    return income_cents - calculate_total_expenses(expenses)
```

to_cents() converts decimal amounts into integer cents and validates inputs; format_money() formats Jamaican dollars; the calculation functions total expenses and compute the balance.

Your task: Create test_finance.py in the project root, paste the following test and run it.

```
from services.finance_service import (
    to_cents,
    format_money,
    calculate_balance
)
expenses = [
    {"amount_cents": to_cents("15000")},
    {"amount_cents": to_cents("5000")}
]
income = to_cents("180000")
balance = calculate_balance(income, expenses)
print(format_money(balance))
python test_finance.py
J$160,000.00
```

```
python test_finance.py
```

Your task: Change the income to J$100,000 and add another expense of J$12,500. Predict the balance before running the test.

Part 4 - Data Persistence with JSON

Persistence means saving information so it can be loaded after a program or session ends. JSON (JavaScript Object Notation) is a widely used text format for structured data. A Python dictionary resembles a JSON object, but they are not identical.

4.1 Initial financial data

Your task: Create data/finances.json and paste this complete JSON object.

```
{
    "income_cents": 0,
    "expenses": []
}
```

4.2 Storage service

Your task: Create services/storage_service.py. Enter the complete code below.

```
{
  "name": "SmartBudget JA",
  "currency": "JMD"
}
{
  "income_cents": 0,
  "expenses": []
}
import json
import os
import tempfile
from pathlib import Path
DATA_DIR = Path("data")
FINANCE_FILE = DATA_DIR / "finances.json"
def load_json(file_path, default):
    path = Path(file_path)
    if not path.exists():
        return default
    with path.open("r", encoding="utf-8") as file:
        return json.load(file)
def save_json(file_path, data):
    path = Path(file_path)
    path.parent.mkdir(
        parents=True,
        exist_ok=True
    )
    temporary_path = None
    try:
        with tempfile.NamedTemporaryFile(
            mode="w",
            encoding="utf-8",
            dir=path.parent,
            suffix=".tmp",
            delete=False
        ) as file:
            temporary_path = file.name
            json.dump(data, file, indent=4)
        os.replace(temporary_path, path)
    finally:
        if temporary_path and os.path.exists(temporary_path):
            os.remove(temporary_path)
def load_finances():
    return load_json(
```

```
        FINANCE_FILE,
        {
            "income_cents": 0,
            "expenses": []
        }
    )
def save_finances(data):
    save_json(FINANCE_FILE, data)
```

The storage service loads and saves JSON separately from the user interface. It writes to a temporary file before replacing the original, reducing the risk of partial writes. This is not a substitute for multi-user database transactions.

4.3 Test persistence

Your task: Create test_storage.py and run the code. Inspect data/finances.json, stop the program and run the test again.

```
from services.storage_service import (
    load_finances,
    save_finances
)
data = {
    "income_cents": 18000000,
    "expenses": [
        {
            "id": "EXP-001",
            "provider": "JPS",
            "category": "Utilities",
            "amount_cents": 1500000,
            "month": "2026-10"
        }
    ]
}
save_finances(data)
saved_data = load_finances()
print(saved_data)
```

```
python test_storage.py
```

Think and discuss: Where is the information stored? How is loading a saved file different from using session state?

Part 5 - Income and Expenses Screen

This screen combines application state, lists and dictionaries, monetary calculations, input validation, unique IDs, and JSON persistence.

Your task: Create screens/finances.py and enter the complete implementation below. The code continues across several blocks; keep them in the same file in order.

```
import uuid
from decimal import Decimal
import streamlit as st
from services.finance_service import (
    to_cents,
    format_money,
    calculate_total_expenses,
    calculate_balance
)
from services.storage_service import (
    save_finances
)
def show_finances():
    st.title("Income & Expenses")
    st.write(
        "Manage your monthly income and expenses."
    )
    data = st.session_state.finances
    # ----------------------------
    # MONTHLY INCOME
    # ----------------------------
    with st.container(border=True):
        st.subheader("Monthly Income")
        income = st.number_input(
            "Net Monthly Salary (J$)",
            min_value=0.0,
            value=data["income_cents"] / 100,
            step=1000.0,
            format="%.2f",
            key="salary_input"
        )
        if st.button("Save Income"):
            data["income_cents"] = to_cents(
                Decimal(str(income))
            )
            save_finances(data)
            st.success("Income saved successfully.")
    # ----------------------------
    # ADD EXPENSE
    # ----------------------------
    with st.container(border=True):
        st.subheader("Add Expense")
        with st.form("expense_form"):
```

```
            provider = st.text_input(
                "Provider / Description"
            )
            category = st.selectbox(
                "Category",
                [
                    "Utilities",
                    "Telecommunications",
                    "Housing",
                    "Groceries",
                    "Transportation",
                    "Credit Card",
                    "Other"
                ]
            )
            amount = st.number_input(
                "Amount (J$)",
                min_value=0.0,
                step=100.0,
                format="%.2f"
            )
            month = st.text_input(
                "Month (YYYY-MM)",
                value="2026-10"
            )
            submitted = st.form_submit_button(
                "Add Expense"
            )
        if submitted:
            import re
            if not provider.strip():
                st.error("Enter a provider or description.")
            elif not re.fullmatch(
                r"\d{4}-(0[1-9]|1[0-2])",
                month
            ):
                st.error("Use a valid YYYY-MM month.")
            elif amount <= 0:
                st.error("Amount must be greater than zero.")
            else:
                expense = {
                    "id": str(uuid.uuid4()),
                    "provider": provider.strip(),
```

```
                "category": category,
                "amount_cents": to_cents(
                    Decimal(str(amount))
                ),
                "month": month
            }
            data["expenses"].append(expense)
            save_finances(data)
            st.success("Expense saved.")
            st.rerun()
# ----------------------------
# EXPENSE LIST
# ----------------------------
st.subheader("Recorded Expenses")
if data["expenses"]:
    for expense in data["expenses"]:
        with st.container(border=True):
            col1, col2 = st.columns([3, 1])
            with col1:
                st.write(
                    f"**{expense['provider']}**"
                )
                st.caption(
                    f"{expense['category']} | "
                    f"{expense['month']}"
                )
            with col2:
                st.write(
                    format_money(
                        expense["amount_cents"]
                    )
                )
                if st.button(
                    "Delete",
                    key=f"delete_{expense['id']}"
                ):
                    data["expenses"] = [
                        item
                        for item in data["expenses"]
                        if item["id"] != expense["id"]
                    ]
                    save_finances(data)
                    st.rerun()
```

```
    else:
        st.info("No expenses recorded yet.")
    # ----------------------------
    # SUMMARY
    # ----------------------------
    st.divider()
    total = calculate_total_expenses(
        data["expenses"]
    )
    balance = calculate_balance(
        data["income_cents"],
        data["expenses"]
    )
    col1, col2, col3 = st.columns(3)
    with col1:
        st.metric(
            "Net Income",
            format_money(data["income_cents"])
        )
    with col2:
        st.metric(
            "Expenses",
            format_money(total)
        )
    with col3:
        st.metric(
            "Remaining",
            format_money(balance)
        )
```

5.1 Enter fictional financial records

Provider	Category	Amount
JPS	Utilities	J$15,000
NWC	Utilities	J$5,000
Flow	Telecommunications	J$7,500
Groceries	Groceries	J$25,000
Rent	Housing	J$45,000
Transport	Transportation	J$15,000

> Your task: Set net monthly income to J$180,000, enter the six expenses, and verify the totals and remaining balance displayed by the application.

Prototype limitation: This version stores one income amount and totals all recorded expenses even if their month fields differ. You will improve this in the independent activity.

Part 6 - Dashboard

Your task: Create screens/dashboard.py and enter the complete code.

```
import streamlit as st
from services.finance_service import (
    calculate_total_expenses,
    calculate_balance,
    format_money
)
def show_dashboard():
    st.title("SmartBudget JA Dashboard")
    st.write(
        "Your personal financial overview."
    )
    data = st.session_state.finances
    income = data["income_cents"]
    expenses = data["expenses"]
    total_expenses = calculate_total_expenses(
        expenses
    )
    balance = calculate_balance(
        income,
        expenses
    )
    col1, col2, col3 = st.columns(3)
    with col1:
        st.metric(
            "Monthly Net Income",
            format_money(income)
        )
    with col2:
        st.metric(
            "Recorded Expenses",
            format_money(total_expenses)
        )
    with col3:
        st.metric(
            "Estimated Balance",
            format_money(balance)
        )
    st.divider()
    if expenses:
        st.subheader("Expenses by Category")
        category_totals = {}
        for expense in expenses:
            category = expense["category"]
```

```
            category_totals[category] = (
                category_totals.get(category, 0)
                + expense["amount_cents"]
            )
        chart_data = {
            category: amount / 100
            for category, amount
            in category_totals.items()
        }
        st.bar_chart(chart_data)
    else:
        st.info(
            "Add expenses to see your dashboard."
        )
```

The dashboard reads the same financial data loaded into session state. The finance service performs calculations; the dashboard focuses on displaying summary metrics and a category chart.

> Your task: Add an expense, navigate to Dashboard, verify updated totals, restart the application and verify saved records are reloaded.

Part 7 - File Handling and Document Uploads

File handling includes opening, reading, writing, storing and managing files. TXT contains plain text; JSON contains structured data; PDFs may contain embedded text or scanned images. Scanned PDFs and images may require OCR to obtain text.

7.1 File uploader experiment

> Your task: Create a temporary upload_demo.py with the code below, run it, and select a test file.

```
import streamlit as st
st.title("File Upload Demonstration")
uploaded_file = st.file_uploader(
    "Choose a file",
    type=["pdf", "png", "jpg", "txt"]
)
if uploaded_file is not None:
    st.write("Filename:", uploaded_file.name)
    st.write("File size:", uploaded_file.size)
    st.write("File type:", uploaded_file.type)
```

> Think and discuss: Before reading a file's contents, what information can you obtain from its filename, size and reported media type? Why should this information not be treated as proof that the file is safe?

7.2 Document service

> Your task: Create services/document_service.py and enter the complete implementation.

```
import uuid
from pathlib import Path
import pymupdf
UPLOAD_DIR = Path("uploads")
ALLOWED_EXTENSIONS = {
    ".pdf",
    ".png",
    ".jpg",
    ".jpeg",
    ".txt"
}
MAX_FILE_SIZE = 5 * 1024 * 1024
def validate_file(uploaded_file):
    extension = Path(
        uploaded_file.name
    ).suffix.lower()
    if extension not in ALLOWED_EXTENSIONS:
        raise ValueError("Unsupported file extension.")
    if uploaded_file.size > MAX_FILE_SIZE:
        raise ValueError(
            "File exceeds the 5 MB limit."
        )
    if uploaded_file.size == 0:
        raise ValueError("File is empty.")
    content = uploaded_file.getvalue()
    if extension == ".pdf":
        if not content.startswith(b"%PDF-"):
            raise ValueError("Invalid PDF signature.")
    elif extension == ".png":
        if not content.startswith(
            b"\x89PNG\r\n\x1a\n"
        ):
            raise ValueError("Invalid PNG signature.")
    elif extension in {".jpg", ".jpeg"}:
        if not content.startswith(b"\xff\xd8\xff"):
            raise ValueError("Invalid JPEG signature.")
    elif extension == ".txt":
        try:
            content.decode("utf-8")
        except UnicodeDecodeError:
            raise ValueError(
                "Text files must use UTF-8."
            )
```

```
    return extension
def save_uploaded_file(uploaded_file):
    extension = validate_file(uploaded_file)
    UPLOAD_DIR.mkdir(
        parents=True,
        exist_ok=True
    )
    safe_filename = (
        f"{uuid.uuid4().hex}{extension}"
    )
    file_path = UPLOAD_DIR / safe_filename
    file_path.write_bytes(
        uploaded_file.getvalue()
    )
    return file_path
def extract_document_text(file_path):
    path = Path(file_path)
    if path.suffix.lower() == ".txt":
        return path.read_text(
            encoding="utf-8"
        )
    if path.suffix.lower() == ".pdf":
        text_parts = []
        with pymupdf.open(path) as document:
            for page in document:
                text_parts.append(
                    page.get_text()
                )
        return "\n".join(text_parts)
    return ""
```

This service checks file extensions, size, and basic signatures, generates a unique filename, saves bytes, and extracts text from TXT or text-based PDF files. These are introductory checks, not full production security validation.

7.3 Documents screen

Your task: Create screens/documents.py and paste the complete code.

```
import streamlit as st
from services.document_service import (
    save_uploaded_file,
    extract_document_text
)
def show_documents():
    st.title("Financial Documents")
    st.write(
        "Upload sample financial documents."
    )
    st.info(
        "Use fictional or redacted documents only."
    )
    uploaded_file = st.file_uploader(
        "Choose a document",
        type=[
            "pdf",
            "png",
            "jpg",
            "jpeg",
            "txt"
        ]
    )
    if uploaded_file is not None:
        st.write(
            "Filename:",
            uploaded_file.name
        )
        st.write(
            "Size:",
            uploaded_file.size,
            "bytes"
        )
        if st.button("Save and Process"):
            try:
                file_path = save_uploaded_file(
                    uploaded_file
                )
                text = extract_document_text(
                    file_path
                )
                st.success(
                    "Document saved successfully."
```

```
            )
            st.write(
                "Stored file:",
                file_path.name
            )
            if text.strip():
                st.subheader("Extracted Text")
                st.text_area(
                    "Document content",
                    value=text,
                    height=250
                )
            else:
                st.warning(
                    "No text was extracted. "
                    "Images and scanned PDFs "
                    "may require OCR."
                )
        except (
            ValueError,
            OSError,
            RuntimeError
        ) as error:
            st.error(str(error))
```

7.4 Create your two sample financial documents

You must create both sample_budget_faq.txt and sample_budget_faq.pdf. Save them in sample_documents/. They must contain the same information.

> Your task: Create sample_budget_faq.txt as a UTF-8 plain-text file. Copy the exact content from the grey reference box below.

```
SMARTBUDGET JA - SAMPLE FINANCIAL GUIDE

Net Pay:
The amount an employee receives after deductions
have been subtracted from gross earnings.

Gross Pay:
The total earnings before deductions.

Utilities:
A budget category that may include electricity
and water expenses.

Budget Balance:
The amount remaining after recorded expenses
have been subtracted from income.
```

> Your task: Create sample_budget_faq.pdf containing the same text. One option is to paste the text into Google Docs, then choose File > Download > PDF Document (.pdf).

Your task: Upload your TXT file in SmartBudget JA, click Save and Process, and inspect the extracted text. Repeat with your PDF file.

Think and discuss: Does extracting text from a document automatically mean you have created a knowledge base? Explain your reasoning.

Part 8 - Understanding and Creating Knowledge Bases

A knowledge base is an organized collection of information used to retrieve facts, explanations, rules or other knowledge. It can be created through manual entry, document ingestion, structured imports, APIs or AI-assisted extraction. Regardless of the method, source reliability, accuracy and maintenance matter.

8.1 Knowledge-base development process

A useful workflow is: define purpose; identify trusted sources; collect information; clean and standardize it; structure records; validate content; store and index it; and test and maintain it.

Think and discuss: Suppose you were creating a knowledge base for a hospital. Would it be acceptable to import random medical advice without checking its source or accuracy? What standards would you apply?

8.2 Create the knowledge records

Your task: Create data/knowledge_base.json. Copy all six knowledge records below into the same JSON array.

```
[
  {
    "id": "KB-001",
    "category": "Income",
    "question": "What is net pay?",
    "answer": "Net pay is the amount of earnings remaining after applicable deductions.",
    "keywords": [
      "net pay",
      "take home pay",
      "salary after deductions"
    ],
    "source": "Instructor-approved glossary"
  },
  {
    "id": "KB-002",
    "category": "Income",
    "question": "What is gross pay?",
    "answer": "Gross pay is the total amount earned before deductions.",
    "keywords": [
      "gross pay",
      "salary before deductions",
      "gross salary"
    ],
    "source": "Instructor-approved glossary"
  },
  {
    "id": "KB-003",
    "category": "Budgeting",
    "question": "What is a budget?",
    "answer": "A budget is a plan for managing income and expenses over a period of time.",
    "keywords": [
      "budget",
      "financial plan",
      "monthly budget"
    ],
    "source": "Instructor-approved glossary"
  },
  {
    "id": "KB-004",
    "category": "Budgeting",
    "question": "What is a budget balance?",
    "answer": "A budget balance is the amount remaining after recorded expenses are subtracted from
the income allocated to the same period.",
    "keywords": [
```

```
      "budget balance",
      "remaining money",
      "income minus expenses"
    ],
    "source": "Instructor-approved glossary"
  },
  {
    "id": "KB-005",
    "category": "Expenses",
    "question": "What are utility expenses?",
    "answer": "Utility expenses commonly include household services such as electricity and water.",
    "keywords": [
      "utilities",
      "electricity",
      "water",
      "utility bills"
    ],
    "source": "Instructor-approved glossary"
  },
  {
    "id": "KB-006",
    "category": "Expenses",
    "question": "What is a recurring expense?",
    "answer": "A recurring expense is a cost that occurs repeatedly, such as monthly rent or an inte
rnet subscription.",
    "keywords": [
      "recurring expense",
      "monthly bills",
      "regular payments"
    ],
    "source": "Instructor-approved glossary"
  }
]
```

Each record includes an ID, category, question, answer, keywords and source. These descriptive fields are metadata; they help organize, retrieve and verify information.

Part 9 - Creating the Knowledge Service

Your task: Create services/knowledge_service.py and enter the complete code below.

```
import re
import uuid
from services.storage_service import (
    load_json,
    save_json
)
KNOWLEDGE_FILE = "data/knowledge_base.json"
def load_knowledge():
    return load_json(
        KNOWLEDGE_FILE,
        []
    )
def normalize_text(text):
    text = text.lower().strip()
    text = re.sub(
        r"[^a-z0-9\s]",
        " ",
        text
    )
    return " ".join(text.split())
def search_knowledge(query):
    records = load_knowledge()
    query = normalize_text(query)
    if not query:
        return []
    query_words = set(query.split())
    results = []
    for record in records:
        question = normalize_text(
            record["question"]
        )
        keywords = [
            normalize_text(keyword)
            for keyword in record["keywords"]
        ]
        searchable_text = " ".join([
            question,
            record["category"],
            *keywords
        ])
        searchable_words = set(
            normalize_text(searchable_text).split()
        )
```

```
        score = len(
            query_words.intersection(
                searchable_words
            )
        )
        if query == question:
            score += 10
        if query in keywords:
            score += 8
        if score > 0:
            results.append({
                "record": record,
                "score": score
            })
    results.sort(
        key=lambda item: item["score"],
        reverse=True
    )
    return results
def add_knowledge(
    category,
    question,
    answer,
    keywords,
    source
):
    if not category.strip():
        raise ValueError("Category is required.")
    if not question.strip():
        raise ValueError("Question is required.")
    if not answer.strip():
        raise ValueError("Answer is required.")
    if not source.strip():
        raise ValueError("Source is required.")
    records = load_knowledge()
    new_record = {
        "id": f"KB-{uuid.uuid4().hex[:8]}",
        "category": category.strip(),
        "question": question.strip(),
        "answer": answer.strip(),
        "keywords": [
            word.strip()
            for word in keywords.split(",")
```

```
            if word.strip()
        ],
        "source": source.strip()
    }
    records.append(new_record)
    save_json(
        KNOWLEDGE_FILE,
        records
    )
    return new_record
```

The search function normalizes text, compares query words with knowledge questions, categories and keywords, assigns match scores, and sorts records by score. It is keyword-based retrieval, not semantic search or RAG.

9.1 Limitations and search improvements

A simple search can match irrelevant common words and cannot automatically understand synonyms. For example, “What is my electricity bill?” may match records only because they contain “what” or “is”.

Stop-word removal filters frequently occurring words such as “what”, “is” and “my”, leaving more informative words. However, removing meaningful words such as “not” can change intent.

```
stop_words = {"what", "is", "my", "the", "a", "an"}
query = "What is my electricity bill?"
words = query.lower().replace("?", "").split()
filtered_words = [
    word for word in words
    if word not in stop_words
]
print(filtered_words)
# ['electricity', 'bill']
```

Phrase matching searches for an ordered sequence such as “electricity bill”, rather than independent words. It improves exactness but misses alternate wording such as “monthly power utility expense”.

```
records = [
    "Electricity bill paid for September",
    "Water bill payment",
    "Monthly power utility expense",
    "Electricity installation at new office"
]
phrase = "electricity bill"
results = [
    record for record in records
    if phrase in record.lower()
]
print(results)
# ['Electricity bill paid for September']
```

Semantic search with embeddings uses a model to represent text as numerical vectors and compare their similarity. It can retrieve conceptually related wording, although high similarity does not guarantee factual correctness. Cosine similarity is a common vector comparison measure.

```
# pip install sentence-transformers
from sentence_transformers import SentenceTransformer
from sentence_transformers.util import cos_sim
model = SentenceTransformer("all-MiniLM-L6-v2")
query = "How much did I pay for electricity?"
records = [
    "Electricity bill paid for September",
    "Water bill payment",
    "Monthly power utility expense",
    "Purchased office stationery"
]
query_vector = model.encode(query)
record_vectors = model.encode(records)
scores = cos_sim(query_vector, record_vectors)[0]
ranked = sorted(
    zip(records, scores.tolist()),
    key=lambda item: item[1],
    reverse=True
)
for record, score in ranked:
    print(f"{score:.3f} - {record}")
```

Optional extension: The embedding example requires sentence-transformers, which is not included in the core requirements. Install it separately if you choose to run the extension:

```
python -m pip install sentence-transformers
```

Think and discuss: How do stop-word removal, phrase matching and semantic search differ? Which technique would you choose for an exact invoice number versus a question expressed in unfamiliar wording?

Part 10 - Knowledge Base Screen

Your task: Create screens/knowledge_base.py and enter the complete code.

```
import streamlit as st
from services.knowledge_service import (
    load_knowledge,
    add_knowledge
)
def show_knowledge_base():
    st.title("SmartBudget Knowledge Base")
    st.write(
        "Explore and manage financial knowledge."
    )
    records = load_knowledge()
    st.metric(
        "Knowledge Records",
        len(records)
    )
    st.divider()
    st.subheader("Available Knowledge")
    if not records:
        st.info("No knowledge records available.")
    for record in records:
        with st.expander(
            record["question"]
        ):
            st.write(record["answer"])
            st.caption(
                f"Category: {record['category']}"
            )
            st.caption(
                f"Source: {record['source']}"
            )
    st.divider()
    st.subheader("Add Knowledge")
    with st.form("knowledge_form"):
        category = st.text_input(
            "Category"
        )
        question = st.text_input(
            "Question"
        )
        answer = st.text_area(
            "Answer"
        )
        keywords = st.text_input(
```

```
            "Keywords (comma separated)"
        )
        source = st.text_input(
            "Information Source"
        )
        submitted = st.form_submit_button(
            "Save Knowledge"
        )
    if submitted:
        try:
            add_knowledge(
                category,
                question,
                answer,
                keywords,
                source
            )
            st.success(
                "Knowledge record saved."
            )
            st.rerun()
        except ValueError as error:
            st.error(str(error))
```

10.1 Add a knowledge record

Field	Value
Category	Banking
Question	What is a credit card minimum payment?
Answer	The minimum payment is the smallest amount the card issuer requires by the payment due date under the card agreement. Paying only the minimum may result in additional interest and a longer repayment period.
Keywords	credit card, minimum payment, card payment
Source	Instructor-reviewed example definition

> Your task: Use the Knowledge Base screen to add this record, save it, restart Streamlit and confirm the record remains available.

The screen provides basic viewing and creation. A production knowledge-management system would also require editing, deletion, review, versioning and access controls.

Part 11 - Ask SmartBudget Screen

This screen retrieves stored knowledge and displays the highest-scoring record. It does not yet call a language model or generate new answers.

> Your task: Create screens/ask_smartbudget.py and enter the complete code.

```
import streamlit as st
from services.knowledge_service import (
    search_knowledge
)
def show_ask_smartbudget():
    st.title("Ask SmartBudget")
    st.write(
        "Ask a question about budgeting "
        "or financial terminology."
    )
    question = st.text_input(
        "Your Question",
        placeholder="What is net pay?"
    )
    if st.button("Find Answer"):
        if not question.strip():
            st.warning(
                "Please enter a question."
            )
        else:
            results = search_knowledge(
                question
            )
            if results:
                best_match = results[0]
                record = best_match["record"]
                st.subheader("Best Matching Answer")
                st.write(
                    record["answer"]
                )
                st.caption(
                    f"Source: {record['source']}"
                )
                st.caption(
                    f"Category: {record['category']}"
                )
                if len(results) > 1:
                    st.subheader(
                        "Other Possible Matches"
                    )
                    for item in results[1:4]:
                        other = item["record"]
                        with st.expander(
```

```
                            other["question"]
                        ):
                            st.write(
                                other["answer"]
                            )
            else:
                st.info(
                    "No matching knowledge was found."
                )
```

A search can produce several ranked results. The top-ranked record is not necessarily correct, so retrieval quality must be evaluated.

Part 12 - Connect the Entire Application

Your task: Create app.py in the project root. Enter the complete code below.

```
import streamlit as st
from services.storage_service import (
    load_finances
)
from screens.dashboard import show_dashboard
from screens.finances import show_finances
from screens.documents import show_documents
from screens.knowledge_base import show_knowledge_base
from screens.ask_smartbudget import show_ask_smartbudget
# ---------------------------------
# APPLICATION CONFIGURATION
# ---------------------------------
st.set_page_config(
    page_title="SmartBudget JA",
    page_icon="",
    layout="wide"
)
# ---------------------------------
# INITIALIZE APPLICATION STATE
# ---------------------------------
if "finances" not in st.session_state:
    st.session_state.finances = (
        load_finances()
    )
# ---------------------------------
# SIDEBAR NAVIGATION
# ---------------------------------
st.sidebar.title("SmartBudget JA")
st.sidebar.caption(
    "Personal Financial Management"
)
st.sidebar.divider()
menu = st.sidebar.radio(
    "Main Menu",
    [
        "Dashboard",
        "Income & Expenses",
        "Documents",
        "Knowledge Base",
        "Ask SmartBudget"
    ]
)
# ---------------------------------
```

```
# SCREEN ROUTING
# ---------------------------------
if menu == "Dashboard":
    show_dashboard()
elif menu == "Income & Expenses":
    show_finances()
elif menu == "Documents":
    show_documents()
elif menu == "Knowledge Base":
    show_knowledge_base()
elif menu == "Ask SmartBudget":
    show_ask_smartbudget()
# ---------------------------------
# APPLICATION FOOTER
# ---------------------------------
st.divider()
st.caption(
    "Amber National AI Lab | "
    "SmartBudget JA Learning Project"
)
```

When Streamlit starts, app.py configures the page, loads saved finances into session state, displays navigation and calls the selected screen. Services keep calculation, storage, document processing and search logic outside the main application file.

Run SmartBudget JA

```
python -m streamlit run app.py
```

Part 13 - Integrated Testing

Test the complete application. Check each item after you have verified it:

- ☐ Application starts without syntax or import errors
- ☐ Income can be entered and saved
- ☐ Multiple expenses can be added and deleted
- ☐ Dashboard totals are correct
- ☐ Saved financial data survives a server restart
- ☐ TXT uploads can be saved and read
- ☐ Text-based PDF content can be extracted
- ☐ Unsupported and oversized files are rejected
- ☐ Knowledge records can be viewed and added
- ☐ Knowledge records survive a restart
- ☐ Ask SmartBudget retrieves matching knowledge
- ☐ Unmatched questions do not crash the app

13.1 Guided test scenarios

Your task: Financial calculation: Enter J$180,000 income and the six sample expenses. Compare the displayed balance with your own calculation.

Your task: Persistence: Stop the Streamlit server completely and restart it. Confirm income and expenses are still present.

Your task: Knowledge retrieval: Search for What is net pay? and take home pay. Compare the retrieved records.

Your task: Missing knowledge: Search for How do I calculate compound interest?. Assess whether any returned result is actually relevant.

Part 14 - Troubleshooting

Problem	What to check
ModuleNotFoundError	Project root, folder names and import paths
KeyError: 'finances'	Session-state initialization in app.py
JSON parsing error	JSON commas, quotes, brackets and field types
Financial data disappears	data/finances.json and current working directory
PDF produces no text	Whether the PDF contains scanned images requiring OCR
Wrong knowledge result	Keywords and limitations of word matching
Button appears not to work	Indentation, conditions and terminal output
Streamlit widget error	Duplicate widget keys
Cannot import pymupdf	Install requirements in the active environment

14.1 Debugging exercise

Your task: Temporarily rename data/knowledge_base.json, open the Knowledge Base screen and observe the behavior. Restore the original filename afterward.

Think and discuss: What is the difference between a missing JSON file and a JSON file whose contents are malformed?

Part 15 - Independent Class Activity

Challenge: Improve SmartBudget JA. Complete these tasks independently. No finished implementation code is provided for these challenges.

Your task: A - Monthly filtering: Add a month selector and show only expenses for the selected month. Clearly address how income relates to the selected month.

Your task: B - Knowledge expansion: Add at least five additional instructor-reviewed financial records, each with a question, answer, category, keywords and source.

Your task: C - Better retrieval: Improve the search function so common words such as “what”, “is” and “the” do not dominate results.

Your task: D - Document organization: Add a document category selector with payslip, electricity bill, water bill, internet bill and other.

Lesson Summary and Next Steps

You have practised session state, Python data structures, exact monetary calculations, JSON persistence, file upload and PDF text extraction, knowledge-base creation and keyword-based retrieval. These form the foundation for a more intelligent application.

Next lesson: SmartBudget JA - From Knowledge Retrieval to RAG. You will explore connecting a local language model, creating embeddings, indexing knowledge in a vector database, retrieving relevant context and generating grounded responses. You will also distinguish RAG from model training and fine-tuning.